**25901183 -Scheme**

# MAHATMA GANDHI UNIVERSITY, KOTTAYAM

## MGU-BCA (HONOURS) REGULAR EXAMINATION March 2026

## Fourth SEMESTER

## Discipline Specific Elective (DSE) - MG4DSEBCA202

## WEB PROGRAMMING - II

**Duration: 1 hour** **Maximum Marks: 35**

***Remember(K), Understand(U), Apply(A), Analyse(An), Evaluate(E), Create(C), Skill(S), Interest(I) and Appreciation(Ap)***

Students should attempt at least one question from each course outcome to enhance their overall outcome attainability.

### Part A

Very Short Answer Questions

Answer any **5** questions Each question carries **3** marks

**1. How do you install a third-party module? [U/CO1]**

Use NPM: `npm install <package-name>` (local, adds to `package.json` dependencies). **(1.5)**

Example: `npm install express` , (For specific version: `npm install express@4.x`) **(1.5)**

**2. What is global installation in NPM? [U/CO1]**

Global installation makes the package available system-wide (usable from any directory via CLI). **(1.5)**

Command: `npm install -g <package-name>` (e.g., `npm install -g nodemon`). Installed in global `node_modules`. **(1.5)**

Use for tools/utilities, not project dependencies. **(1.5)**

***[Give full marks for any 2 points]***

**3. What is thread-based request handling? [U/CO1]**

Traditional web servers (e.g., Apache) use a thread-per-request model: each request spawns a new thread, which can lead to high memory use and context-switching overhead under load. Node.js uses a single-threaded, event-driven, non-blocking model instead.

***[Concept 2 marks + example 1 mark]***

4. **Describe the purpose of req and res objects in HTTP communication. [U/CO2]**

**req(Request)**: Represents the incoming HTTP request from the client. Contains properties like `req.url`, `req.method`, `req.headers`, `req.params`, `req.query`, `req.body`.

***[Concept 1 mark + any one property .5 mark]***

**res (Response)**: Used to send the HTTP response back to the client. Methods like `res.send()`, `res.json()`, `res.status()`, `res.end()`.

***[Concept 1 mark + any one property .5 mark]***

5. **Differentiate between Node.js runtime and Express framework. [A/CO3]**

| **Node.js** | **Express** |
|---|---|
| A JavaScript runtime environment. | A web application framework. |
| C, C++, and Chrome's V8 Engine. | Built entirely on top of Node.js. |
| Executing JS code on the server side. | Building web applications and RESTful APIs easily. |
| Complex to set up manually using if/else statements checking the URL. | Highly intuitive and built-in (e.g., app.get('/about', ...)). |
| Does not natively support middleware architecture. | Middleware is a core feature for handling authentication, logging, etc. |
| Requires more lines of code for basic web server tasks. | Requires far fewer lines of code; abstracts away the complex parts. |

***[Each point give .5 marks; any 3 comparison give full marks]***

6. **Why is EventEmitter important in asynchronous programming? [A/CO3]**

`EventEmitter` (from `events` module) enables event-driven architecture. **(1)**

Emit custom events (`emitter.emit('event')`) and listen (`emitter.on('event', callback)`). **(1)**

It supports loose coupling, handles async operations (e.g., file read completion, DB events) without blocking, and is core to Node.js's non-blocking model. **(1)**

7. **What is blocking operation? [A/CO3]**

A blocking (synchronous) operation halts the event loop until it completes

e.g., `fs.readFileSync()`

No other JS code runs during this time, reducing concurrency.

Node.js prefers non-blocking async alternatives (callbacks/Promises) for scalability

***[Any 3 points give full marks]***

**Part B: Short Answer Questions (Answer any 4, 5 marks each)**

**8. Explain how Function Scope and Global Scope behave in Node.js compared to a browser environment [U/CO1]**

In JavaScript, scope defines the accessibility of variables. There are two main scopes to compare between Browser and Node.js environments:

**1. Global Scope**

In Browser Environment: Any variable declared with `var` outside a function becomes a global variable and is attached to the `window` object.

*Example:*

```
var x = 10;     // Global
console.log(window.x); // 10
```

All scripts in the same HTML page share the same global scope.

In Node.js Environment: Each JavaScript file is treated as a separate module. Variables declared at the top level (using `var`, `let`, or `const`) are local to that module (Module Scope), not global. They are not added to the `global` object.

*Example*:

```
var x = 10;     // Module scoped (only available in this file)
console.log(global.x); // undefined
```

To create a true global variable in Node.js, we must explicitly use:

```
global.myVar = "Hello";   // True global
```

**2.Function Scope**

In both Browser and Node.js: Variables declared with `var` inside a function have function scope (accessible anywhere inside that function, even before declaration due to hoisting).

`let` and `const` have block scope in both environments.

The main difference appears in top-level code. In Node.js, top-level code is wrapped inside a module wrapper function, so it behaves like function scope.

*[Introduction to Scope + mentioning Global Scope difference – 2 marks. Detailed explanation of Global Scope in Browser vs Node.js with examples – 2 marks. Function Scope explanation (similarity + top-level difference) – 1 mark]*

**9. Explain async/await with advantages. [U/CO1]**

async/await is a modern feature in JavaScript (introduced in ES2017) used to handle asynchronous operations in a cleaner way. It allows us to write asynchronous code that appears more like synchronous code.

**How async/await Works:**

1. **async keyword**: Used before a function declaration. It tells JavaScript that the function always returns a **Promise**.
2. **await keyword**: Can only be used inside an async function. It **pauses** the execution of the function until the Promise is resolved or rejected.

*Example:*

```
async function getUserData() {
   try {
      let response = await fetch('https://api.example.com/user');
      let user = await response.json();
      console.log(user);
   }
   catch (error) {
      console.log("Error occurred:", error);
   }}
getUserData();
```

**Advantages of async/await:**

- Improved Readability: The code looks simple and linear, making it easy to understand the flow.
- Better Error Handling: We can use the normal try...catch block for error handling.
- Easier Debugging: Debugging is simpler compared to callbacks or promise chains.
- Sequential Execution: Helps to write asynchronous operations in a step-by-step manner.
- Less Complex Code: Reduces nesting and makes the code more maintainable.

*[Definition of async/await -1 mark + Explanation of async and await keywords2 Marks + Clear code example 2 Marks + At least 4 advantages -2 marks: Maximum 5 Marks]*

**10. Explain the importance of directory management in server-side programming. [U/CO2]**

Directory management is a crucial part of server-side programming, especially in Node.js using the **File System (fs)** module. It involves creating, reading, updating, and deleting directories on the server.

**Importance of Directory Management:**

1. **Organized File Storage**: Proper directory structure helps to organize different types of files such as user uploads, log files, configuration files, static assets (images, CSS, JS), and temporary files. This makes the application cleaner and easier to maintain.
2. **User Data Management**: Servers often need to store user-specific data (e.g., profile pictures, documents). Dynamic directory creation allows each user to have their own folder, improving data organization and security.
3. **Application Scalability**: As the application grows, directory management helps in handling large volumes of files efficiently without cluttering the root folder.

4. **Temporary File Handling**: Many server operations require temporary directories for processing files (e.g., file uploads, report generation). These can be created and deleted after use.
5. **Security**: Proper directory management allows setting correct permissions, restricting access to sensitive folders, and preventing unauthorized file access.
6. **Integration with Web Features**: Directories are essential for serving static files (using express.static()), managing templates, and working with URL modules for file routing.

In Node.js, functions like fs.mkdir(), fs.readdir(), fs.rmdir(), and fs.promises are used for directory operations.

*[Introduction to directory management in server-side (Node.js) 1mark + Explanation of importance (any 4 points) 3 Marks + fs module functions(any 2) – 1mark]*

**11. Describe file reading and writing mechanisms in Node.js. [U/CO2]**

Node.js provides file handling operations through the built-in fs (File System) module. There are two main mechanisms for reading and writing files:

1. Synchronous Mechanism (Blocking)

- Operations are performed synchronously. The program waits until the file operation is completed.
- Not recommended for web servers as it blocks the event loop.

*Example*

```
const fs = require('fs');
// Reading a file
const data = fs.readFileSync('file.txt', 'utf8');
console.log(data);
// Writing a file
fs.writeFileSync('file.txt', 'Hello World');
```

**2. Asynchronous Mechanism (Non-Blocking) - Recommended**

- Operations are performed **asynchronously** using callbacks or promises.
- The program does not wait and continues executing other code.

**Examples:**

*Reading a file asynchronously:*

```
const fs = require('fs');
fs.readFile('file.txt', 'utf8', (err, data) => {
   if (err) throw err;
   console.log(data);
});
```

*Writing a file asynchronously:*

```
fs.writeFile('file.txt', 'Welcome to Node.js', (err) => {
    if (err) throw err;
    console.log('File written successfully');
});
```

*[Introduction to fs module + Synchronous vs Asynchronous 2 Marks, Explanation with code examples of reading and writing (Sync + Async) 2 Marks, Advantages of Async over Sync – 1 Mark]*

**12. Demonstrate the various methods provided by the Express.js Response (res) object. Include examples for sending JSON, setting status codes, and redirecting requests. [A/CO3]**

The **res** (Response) object in Express.js is used to send responses back to the client. It provides many useful methods to control the HTTP response.

**Important Methods of res Object:**

1. **res.send()** Sends a response of various types (string, object, array, etc.).
2. **res.json()** — Sends a JSON response. Automatically sets the Content-Type header to application/json.

   *Example:*

   ```
   app.get('/api/user', (req, res) => {
       res.json({ name: "Biju", age: 22, city: "Kollam" });
   });
   ```
3. **res.status()** — Sets the HTTP status code for the response.

   *Example:*

   ```
   app.get('/notfound', (req, res) => {
       res.status(404).send("Page Not Found");
   });
   ```
4. ***res.*redirect*()*** *— Redirects the client to a different URL.*

   *Example:*

   ```
   app.get('/old-page', (req, res) => {
       res.redirect('/new-page');          // Default 302 redirect
       // res.redirect(301, '/new-page');  // Permanent redirect
   });
   ```

**Other Useful Methods:**

- res.sendFile() → Send a file (e.g., image, PDF)
- res.render() → Render template engines (like Pug/EJS)

- res.end() → End the response process

*[Introduction to res object -1mark, Explanation + examples any 4 methods -4 marks]*

**13. Explain how to delete records in Node.js. [U/CO3]**

In Node.js, deleting records is done by connecting to a database and executing a **DELETE** query. Node.js itself does not have built-in database support, so use database drivers like mysql2.

**Using MySQL**

```
const mysql = require('mysql2');
const connection = mysql.createConnection({
   host: 'localhost',
   user: 'root',
   password: 'password',
   database: 'mydb'
});

connection.connect();

// Delete a single record
const sql = "DELETE FROM users WHERE id = ?";
const id = 5;

connection.query(sql, [id], (err, result) => {
   if (err) throw err;
   console.log("Number of records deleted: " + result.affectedRows);
});
```

*[Introduction (Need of database driver + DELETE operation) -1 Mark + MySQL Delete example with code -4 marks]*